

Containerized React + Django Monorepo

ARCHITECTURE

Containerized Monorepo

INITIAL STATE

Minimal / Zero External Dependencies

STATUS

Base Specification (Draft)

1. Overview & System Design

This document outlines the core technical architecture, stack decisions, and environment setup for the project. The application is structured as a single repository (monorepo) comprising a React frontend (`client/`) and a Django backend (`server/`), orchestrated using Docker containers.

Design Philosophy: Zero-Dependency Baseline

The project initial phase maintains a lean dependency footprint. Core frameworks (React and Django) are utilized standard out-of-the-box before introducing heavy third-party packages or complex state managers.

2. Core Technology Stack

Layer	Technology	Version Target	Purpose / Role
Frontend	React	18.x / 19.x	UI library for building component-based client interface.
Backend Framework	Django	5.x	Robust Python web framework for core logic, ORM, and API endpoints.
REST API	Django REST Framework	3.15.x	Serialization, authentication, and endpoint routing (optional initial dependency).
Database	PostgreSQL	16.x (Alpine)	Relational database storage engine running in Docker.
Containerization	Docker & Compose	v2.x+	Isolated environment execution and local orchestration.
Web Server / Proxy	Nginx (Production)	Mainline Alpine	Reverse proxy for routing request traffic and serving static assets.

3. Repository Structure

```
.
├── docker-compose.yml
├── docker-compose.override.yml
├── .gitignore
├── .env.example
├── client/                    # React Frontend App
│   ├── Dockerfile
│   ├── package.json
│   ├── public/
│   └── src/
└── server/                   # Django Backend App
    ├── Dockerfile
    └── manage.py
```

```
|— requirements.txt
|— core/                # Main Django config module
```

4. Container & Orchestration Topology

The local development environment uses `docker-compose` to bridge services across a shared bridge network, enabling standard service-name DNS discovery.

Service Name	Container Port	Host Port Mapping	Volume Mounts
<code>client</code>	3000	<code>3000:3000</code>	<code>./client:/app</code> (Live reloading)
<code>server</code>	8000	<code>8000:8000</code>	<code>./server:/app</code> (Live reloading)
<code>db</code>	5432	<code>5432:5432</code>	<code>postgres_data:/var/lib/postgresql/data</code>

5. Security & Environment Configuration

Environment Variables (.env)

Never commit actual `.env` files to version control. Maintain key names and default local dev values strictly in `.env.example`.

- **Django Secret Key:** Kept strictly inside environment variables.
- **CORS Headers:** Django configured via `django-cors-headers` to accept requests originating from `http://localhost:3000` during local dev.
- **Database Credentials:** Shared secret values injected into both `db` and `server` containers via root `.env` file.

6. Future Expansion Roadmap

As features require additional tooling, the following dependencies are slated for potential integration:

- **State Management:** Redux Toolkit or Zustand (if local React state becomes unmanageable).
- **Background Jobs:** Celery + Redis for asynchronous task processing.
- **Authentication:** JWT (SimpleJWT) or Session-based auth via DRF.
- **Styling / UI:** Tailwind CSS or Material UI / Shadcn for rapid interface development.